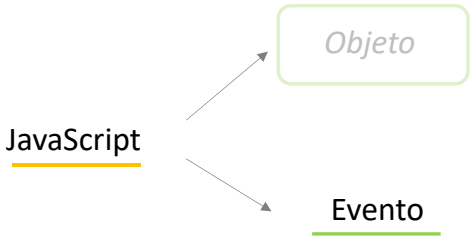
Eventos

Eventos



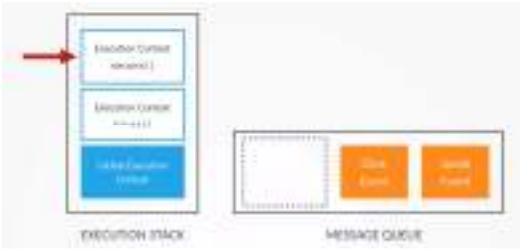
Evento	É disparado...
click	quando é pressionado e liberado o botão primário do mouse, trackpad, etc.
mousemove	sempre que o cursor do mouse se move.
mouseover	quando o cursor do mouse é movido para sobre algum elemento.
mouseout	quando o cursor do mouse se move para fora dos limites de um elemento.
dbiclick	quando acontece um clique duplo com o mouse, trackpad, etc.

Eventos

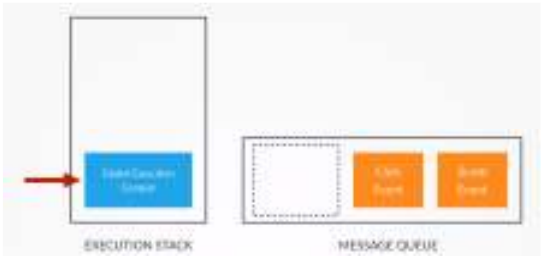
- É necessário um identificador (*event handler*) que permita iniciar (*trigger*) a execução do *script* quando esse evento ocorre
  - Existem diferentes tipos de eventos:
    - User Interface Events
    - HTML elements Events
    - Mouse events
    - Keyboard events
    - Mutation events
    - …

http://www.w3schools.com/jsref/dom_obj_event.asp

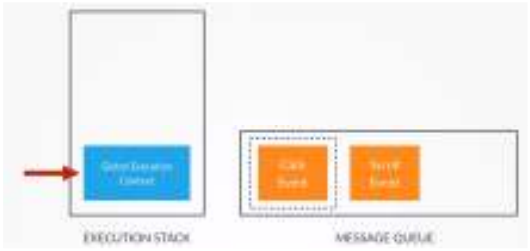
Processamento de Eventos



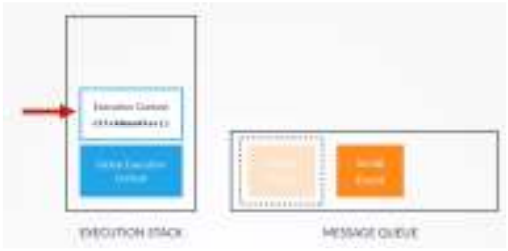
1. Enquanto funções estiverem a ser executadas, os eventos não podem ser considerados;



2. Uma vez libertados os contextos de execução (terminada a execução das funções), os eventos podem ser considerados



3. Quando um determinado evento ocorre é processado no *message queue*



4. É então criado o seu próprio contexto de execução (processamento/função associado à deteção do evento)

https://www.udemy.com/the-complete-javascript-course/learn/v4/t/lecture/5869158?start=0

Eventos

- *Event handling*

1. Seleção do elemento HTML onde é definido o evento
   - Os eventos relacionados com o *Window Object* não se referem à DOM tree.
2. Definir o evento que faz o *trigger* da ação pretendida
   - Alguns eventos podem ser aplicados à maioria dos elementos
     - Exemplo: *mouseover*, …
   - Outros eventos apenas podem ser definidos em elementos específicos
     - Exemplo: *submit*, …
3. Definir o processamento
   - Código necessário para executar uma determinada ação

Eventos

- Três formas distintas de declarar um evento:

1. *HTML Event Handlers*
   - Usa diretamente atributos dos elementos HTML para responder a eventos no elemento onde foram declarados.
   - Considerada **má prática** uma vez que associa de forma direta o HTML e o JavaScript
     - Deve existir uma separação clara na aplicação das duas tecnologias

*<a **on**click="hide()"> … *

2. *DOM Event Handlers*
   - Considerados melhores que a opção anterior uma vez que permitem uma separação clara entre *Javascript* e HTML
   - A limitação principal é que permite associar apenas uma função por evento e não permite a passagem de valores à função

*element.**on**event=functionName;*

3. *DOM Event Listeners*
   - sintaxe diferente
   - Permitem associar múltiplas funções ao mesmo evento e passagem de argumentos

element.addEventListener('event',functionName,[Boolean]);

Eventos

- *HTML Event Handler*
  - **Não permite** uma separação clara entre o HTML e o *Javascript*

```
<form method="post" action="#">
    <label for="username">username:</label>
    <input type="text" id="username" onblur="checkUsername()" />

    <div id="feedback"></div><br />

    <label for="password">password:</label>
    <input type="password" id="password" />

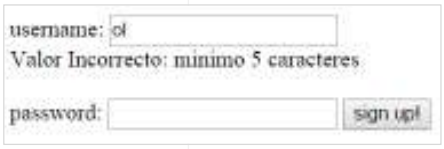
    <input type="submit" value="sign up!" />
</form>

<script>

    function checkUsername() {
        var elMsg = document.getElementById('feedback');
        var elUsername = document.getElementById('username');

        if (elUsername.value.length < 5) {
            elMsg.textContent = 'Valor Incorrecto: minimo 5 caracteres';
        } else { elMsg.textContent = '';}
    }
</script>
```

Quando o utilizador seleciona outro campo é disparado o evento



Testa o número de caracteres

Eventos

- *DOM Event handler*
  - Permite uma separação clara entre o HTML e o *Javascript (o evento é definido no <script>)*

*element.**on**event=functionName*

```
<form method="post" action="#">
    <label for="username">username:</label>
    <input type="text" id="username" />

    <div id="feedback"></div><br />

    <label for="password">password:</label>
    <input type="password" id="password" />

    <input type="submit" value="sign up!" />
</form>

<script>
    function checkUsername() {
            var elMsg = document.getElementById('feedback');
            var elUsername = document.getElementById('username');

            if (elUsername.value.length < 5) {
                elMsg.textContent = 'Valor Incorrecto: minimo 5 caracteres';
            } else { elMsg.textContent = ''; }
    }
    var elUser = document.getElementById("username");
    elUser.onblur = checkUsername;
</script>
```

Evento não é definido no HTML



Evento definido no JS (não permite a passagem de parâmetros, apenas o nome da função)

Eventos

- *Event Listener*
  - Podem invocar mais do que uma função mas não são suportados pelos *browsers* mais antigos

element.addEventListener('event',functionName,[Boolean])

```
<form method="post" action="#">
    <label for="username">username:</label>
    <input type="text" id="username" />

    <div id="feedback"></div><br />

    <label for="password">password:</label>
    <input type="password" id="password" />

    <input type="submit" value="sign up!" />
</form>


<script>
    function checkUsername() {
        var elMsg = document.getElementById('feedback');
        var elUsername = document.getElementById('username');

        if (elUsername.value.length < 5) {
            elMsg.textContent = 'Valor Incorrecto: minimo 5 caracteres';
        } else { elMsg.textContent = ''; }
    }

    var elUser = document.getElementById("username");
    elUser.addEventListener('blur', checkUsername, false);
</script>                    *        **       ***
```

Evento não é definido no HTML

* Ao contrário das duas situações anteriores o evento não é precedido de 'on'

** apenas o nome da função, os parêntesis são omitidos.

*** Quando existem eventos encadeados define a ordem dos eventos. Valor por defeito *false (bubling)*

Eventos

- Passagem de Valores a uma Função
  - *Event handlers & Listeners*
    - Solução passa por definir uma ***anonymous function*** tal que permita receber argumentos
    - A *anonymous function* pode conter várias chamadas a funções diferentes.

```
<script>
    function checkUsername(minlength) {
        var elMsg = document.getElementById('feedback');
        var elUsername = document.getElementById('username');

        if (elUsername.value.length < minlength) {
            elMsg.textContent = 'Valor Incorrecto: minimo 5 caracteres';
        } else { elMsg.textContent = ''; }
    }

    var elUser = document.getElementById("username");
    elUser.addEventListener('blur', function(){ checkUsername(5);} , false);
</script>
```

Com base numa ***anonymous function*** é passado o valor 5 à função *checkUsername*

Event Object

HTML DOM Reference

The references describe the properties and methods of each object, along with examples.

Attributes	Console	Document	Element	Events
Event Objects	Geolocation	History	HTMLCollection	Location
Navigator	Screen	Style	Window	Storage

Event Object

- Quando ocorre um evento, é criado automaticamente um ***event object***
  - Guarda informação sobre o evento (tipo, elemento onde ocorreu o evento, …)
  - É passado automaticamente a qualquer função que seja ***event handler*** **ou** ***listener***
    - A função é definida geralmente com o parâmetro e (*event*)

Property	Description
bubbles	Returns whether or not a specific event is a bubbling event
cancelable	Returns whether or not an event can have its default action prevented
target	Returns the element that triggered the event
timeStamp	Returns the time (in milliseconds relative to the epoch) at which the event was created
type	Returns the name of the event
view	Returns a reference to the Window object where the event occured

Method	Description
preventDefault()	Cancels the event if it is cancelable, meaning that the default action that belongs to the event will not occur
stopImmediatePropagation()	Prevents other listeners of the same event from being called
stopPropagation()	Prevents further propagation of an event during event flow

http://www.w3schools.com/jsref/dom_obj_event.asp

Event Object

- *O event object* é passado de forma automática à função que é disparada pelo evento
  - tipicamente identifica-se por (e) (não é obrigatório!)
  - permite utilizar as propriedades e métodos do *event object* no interior da função

```
<form method="post" action="#">
<label for="username">username:</label>
<input type="text" id="username" />

<div id="feedback"></div><br />

<label for="password">password:</label>
<input type="password" id="password" />

<input type="submit" value="sign up!" />
</form>
```



Event Object

- Função executada **não possui** parâmetros de entrada

```
<div id="evObj"></div>

<script>
    function checkUsername(e) {
        var elTargetUser = e.target;
        document.getElementById('evObj').innerHTML = elTargetUser;
    }

    var elUser = document.getElementById("username");
    elUser.addEventListener('blur', checkUsername, false);
</script>
```

Propriedades e métodos disponíveis no event (ex: target)

Evento disparado, o event object é passado à função

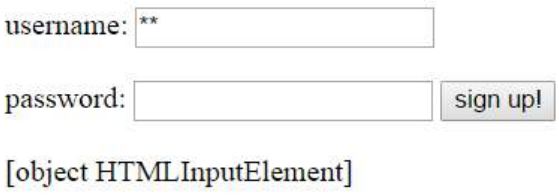


Event Object

- Função executada **possui** parâmetros de entrada

```
<div id="evObj"></div>

<script>
    function checkUsername(e, minLength) {
        var elTarget = e.target;
        document.getElementById('evObj').innerHTML = elTarget;
    }
    var elUser = document.getElementById('username');
    elUser.addEventListener('blur', function (eObj){checkUsername(eObj, 5);}, false);
</script>
```



Tipos de Eventos

Tipos de Eventos

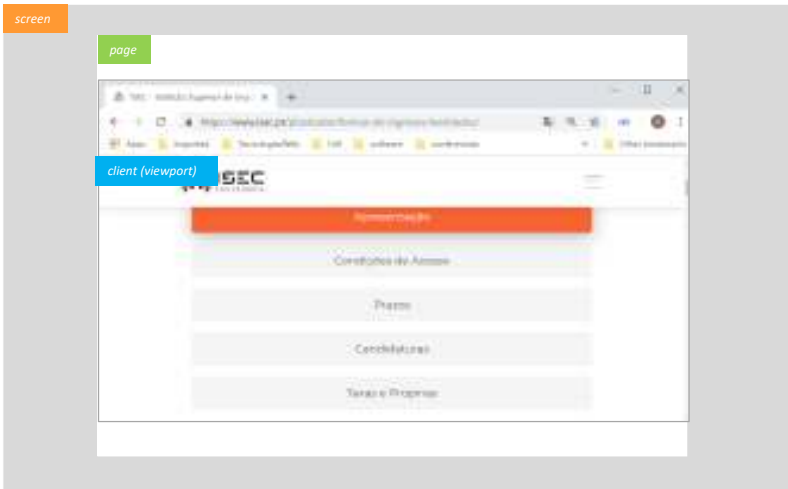
- *Mouse Events*
  - Dispara quando o:

event	trigger
click	utilizador clica no botão esquerdo do rato. Também dispara quando o utilizador pressiona a tecla *Enter* num elemento com *focus*
dbclick	utilizador faz um *double click* no botão esquerdo do rato
mousedown	utilizador pressiona qualquer botão do rato
mouseup	utilizador liberta qualquer botão do rato (não pode ser iniciado por teclado)
mouseover	cursor do rato é sobreposto a um determinado elemento
mouseout	cursor do rato está sobre um elemento e se move para outro elemento (externo ao elemento inicial / não é filho do elemento inicial)
mousemove	cursor é movido em torno de um elemento (este evento é disparado de forma repetida)

Mouse Events

Exemplo

Propriedades específicas mouse events



screen

screenX *e* ***screenY*** *indicam a posição relativamente à totalidade do ecrã (top lef corner)*

page

pageX *e* ***pageY*** *indicam a posição do cursor relativamente à página.*
O topo da página pode estar fora do viewport, ou seja mesmo com o cursor na mesma posição as coordenadas podem ser diferentes

client

clientX *e* ***clientY*** *indicam a posição do cursor relativamente ao viewport. O scroll da página não afeta estas coordenadas*

Mouse Events (propriedades específicas)

```
<ul id="posicao">
    <li id="sx"></li>
    <li id="sy"></li>
    <li id="px"></li>
    <li id="py"></li>
    <li id="cx"></li>
    <li id="cy"></li>
</ul>

<script>
    var sx = document.getElementById('sx');
    var sy = document.getElementById('sy');
    var px = document.getElementById('px');
    var py = document.getElementById('py');
    var cx = document.getElementById('cx');
    var cy = document.getElementById('cy');

    function mostraPosicao() {
        sx.innerHTML = event.screenX;
        sy.innerHTML = event.screenY;
        px.innerHTML = event.pageX;
        py.innerHTML = event.pageY;
        cx.innerHTML = event.clientX;
        cy.innerHTML = event.clientY;}

    var elEvento = document.getElementById('posicao');
    elEvento.addEventListener('mousemove', mostraPosicao, false);
</script>
```

screen

page

client

Posição do cursor do rato

- 109
- 153
- 87
- 54
- 87
- 54

Tipos de Eventos

- *User Interface Events*
  - Dispara:

event	*trigger*
load	quando é finalizado o download da página. Também pode ser aplicado a outros elementos em que é necessário o download, como *images, scripts, objects*
error	quando o browser encontra um erro JavaScript
resize	quando a janela do browser foi redimensionada
scroll	quando o user faz um scroll (down/up) da página

User Interface Events

- *Exemplo: load event*

```
<form method="post" action="#">
    <label for="username">username:</label>
    <input type="text" id="username" />

    <div id="feedback"></div><br />

    <label for="password">password:</label>
    <input type="password" id="password" />

    <input type="submit" value="sign up!" />
</form>


<script>

    function loadEvent() {
        var elFocus = document.getElementById('password');
        elFocus.focus();
    }
    window.addEventListener('load', loadEvent, false);
</script>
```



*Evento **'load'***

Tipos de Eventos

- *Keyboard Events*
  - Dispara quando o:

event	trigger
input	conteúdo de um elemento input ou textarea é alterado
keydown	utilizador pressiona qualquer tecla do teclado. Se o utilizador mantiver a tecla pressionada o evento dispara de forma repetida.
keypress	dutilizador pressiona qualquer tecla que resultaria num caracter visível no ecrã.
keyup	utilizador liberta uma tecla do teclado. Este evento, ao contrário de keydown e keypress, é disparado após a representação do caracter.

Keyboard Events (propriedades específicas)

KeyboardEvent Object

Property	Description
altKey	Returns whether the "ALT" key was pressed when the key event was triggered
ctrlKey	Returns whether the "CTRL" key was pressed when the key event was triggered
charCode	Returns the Unicode character code of the key that triggered the onkeypress event
key	Returns the key value of the key represented by the event
keyCode	Returns the Unicode character code of the key that triggered the onkeypress event, or the Unicode key code of the key that triggered the onkeydown or onkeyup event
location	Returns the location of a key on the keyboard or device
metaKey	Returns whether the "meta" key was pressed when the key event was triggered
shiftKey	Returns whether the "SHIFT" key was pressed when the key event was triggered
which	Returns the Unicode character code of the key that triggered the onkeypress event, or the Unicode key code of the key that triggered the onkeydown or onkeyup event

Tipos de Eventos

```
<form action="#">
    <textarea rows="5" cols="20" id="message"></textarea>
</form>
<div id="charleft"></div>
<div id="lastkey"></div>

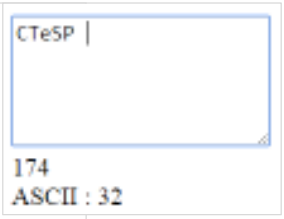

<script>
    function contaCaracteres(e) {

        var textEnter = document.getElementById('message').value;
        counter = (180 - (textEnter.length));

        var charDisplay = document.getElementById('charleft');
        charDisplay.textContent = counter;

        var lastkey = document.getElementById('lastkey');
        lastkey.textContent = 'ASCII   :    ' + e.keyCode;
    }

    var elTextarea = document.getElementById('message');
    elTextarea.addEventListener('keypress', contaCaracteres, false);
</script>
```



2) atualiza o contador

3) mostra o nº de caracteres em falta

4) código ASCII ultima tecla

1) disparado repetidamente

Tipos de Eventos

- *Form Events*

event	Trigger
submit	dispara no nó que representa o elemento *form*, geralmente utilizado para verificar os valores introduzidos antes de enviar ao servidor
change	dispara quando uma alteração é efetuada em alguns elementos do *form* (ex: seleção num menu *drop-down*, seleção de um *radio button*, seleção de uma *check box*, …)
input	dispara quando são introduzidos valores nos elementos *input* e/ou *textarea*
blur	dispara quando se abandona um determinado elemento. Não é exclusivo dos *forms* (pode por exemplo ser utilizado com *links*).
focus	dispara quando se seleciona um determinado elemento. Não é exclusivo dos *forms* (pode por exemplo ser utilizado com *links*).

Tipos de Eventos

- *Mutation Events*
  - Sempre que a estrutura da *DOM tree* é alterada (elementos adicionados ou removidos) é disparado um *mutation event*

event	Trigger
DOMNodeInserted	dispara quando um nó é inserido na DOM Tree
DOMNodeRemoved	dispara quando um nó é removido da DOM Tree
DOMSubtreeModified	dispara quando a estrutura da DOM Tree é alterada.
DOMNodeInsertedIntoDocument	dispara quando um nó é inserido na DOM Tree como descendente de outro nó já existente
DOMNodeRemovedFromDocument	dispara quando um nó é removido da DOM Tree como descendente de outro nó já existente

Mutation Events

```
<h2>Lista de Compras <span id="counter">1</span></h2>
<ul id="list">
  <li>morangos</li>
</ul>
<div><a href="#">Novo list item</a></div>

 <script>
     var elList = document.getElementById('list');
     var elLink = document.getElementsByTagName('a')[0];
     var elCounter = document.getElementById('counter');

     function adicionaElemento() {

         var elNewtext = document.createTextNode('café');
         var elNew = document.createElement('li');

         elNew.appendChild(elNewtext);
         elList.appendChild(elNew);}

     function atualizaContador() {
         var elDim = document.getElementsByTagName('li').length;
         elCounter.innerHTML = elDim;}

     elLink.addEventListener('click', adicionaElemento, false);
     elList.addEventListener('DOMNodeInserted', atualizaContador, false);
 </script>
```

Lista de Compras 1

- morangos

Novo list item

Lista de Compras 2

- morangos
- café

Novo list item

é disparado a cada alteração

Tipos de Eventos

- *Other HTML Events*
  - Eventos disponibilizados pela última versão do HTML

Event	Trigger
DOMContentLoaded	dispara no window object quando a DOM Tree está formada (efetuado download dos elementos HTML), ou seja o script é executado antes de ter sido efetuado o download de todos os outros elementos (css, imagens, …).
beforeunload	dispara no window object antes de se efetuar o unload da página (deve constituir uma ajuda ao utilizador, por exemplo informar o utilizador de que as alterações a um formulário não foram gravadas).

FIM!